

Software Engineering - Lab 7: Requirement Analysis & Specification

Overview

In our previous lectures, we discussed the Requirement Engineering process:

Elicitation - Analysis - Specification - Validation - Management.

For this assignment, you will step into the role of a **Requirement Analyst**. You have completed the *Elicitation* phase (interviews, surveys, and observation) for three potential university-based software projects. You are now left with a collection of "Raw Elicitation Notes", a mix of stakeholder requests, constraints, and vague ideas.

Your goal is to take this raw information, **Analyze** it for clarity and feasibility, and produce a formal **Specification** using User Stories.

Learning Objectives

By completing this assignment, you will be able to:

1. Analyze raw data to identify functional and non-functional requirements.
2. Resolve ambiguities and conflicts within stakeholder statements.
3. Translate requirements into standard User Stories with Acceptance Criteria.

Prerequisites

1. A modern web browser (Chrome, Edge, or Firefox recommended).
2. Git installed locally and a GitHub account.
3. Basic familiarity with user story syntax and acceptance criteria.
4. The project starter package provided in this repository for your assigned project repository (project-a, project-b, or project-c).

Submission Guidelines

Please adhere strictly to the following guidelines. Failure to follow format instructions may result in a deduction of points.

- **The URL to your public GitHub repository:**
`https://github.com/<your-username>/se-req-<StudentID>-<YourName>`
- Commit messages must include the exact grading identifiers listed in this document. Identifiers are case-sensitive.

Important: Before every commit that is used for grading, click Save in the board UI and commit the exported JSON file (recommended path: `board/submission.json`).

Instructions

Part A: Requirement Analysis

Part A for project-a

Open `project-a/board/index.html`, switch to Part A: Analysis, and read `project-a/docs/raw-elicitation-notes.md`. Complete all fields for **project-a** in the order below.

A.1 Ambiguity Check (2 required submissions)

Identify two requirements that are currently too vague to build. Explain why they are vague.

1. Complete "**Ambiguity 1 - Explanation**". Save JSON to the project board folder (`submission.json`).
2. Commit with identifier **A-project-a-AmbiguityCheck-1**:

```
git add project-a/board/submission.json
git commit -m "A-project-a-AmbiguityCheck-1: document first ambiguous requirement"
```

3. Complete "**Ambiguity 2 - Explanation**". Save JSON again.
4. Commit with identifier **A-project-a-AmbiguityCheck-2**:

```
git commit -m "A-project-a-AmbiguityCheck-2: document second ambiguous requirement"
```

A.2 Conflict Resolution (1 required submission)

Identify any conflicting requests between stakeholders (e.g., User A wants X, but User B wants Y). Propose a solution.

1. Complete the **Conflict Resolution** field. Save JSON.
2. Commit with identifier **A-project-a-ConflictResolution**:

```
git commit -m "A-project-a-ConflictResolution: resolve stakeholder conflict with proposed solution"
```

A.3 Categorization: Top 3 Non-Functional Requirements

List the top 3 Non-Functional Requirements (NFRs) derived from the notes (e.g., security, performance, usability).

1. Complete NFR 1, 2, and 3. Save JSON.
2. After all NFR fields for **project-a** are complete:

```
git commit -m "A-project-a-Categorization: define 3 NFRs"
```

Part A for project-b

Open `project-b/board/index.html`, switch to Part A: Analysis, and read `project-b/docs/raw-elicitation-notes.md`. Complete all fields for project-b in the order below.

A.1 Ambiguity Check (2 required submissions)

Identify two requirements that are currently too vague to build. Explain why they are vague.

1. Complete "**Ambiguity 1 - Explanation**". Save JSON to the project board folder (`submission.json`).
2. Commit with identifier **A-project-a-AmbiguityCheck-1**:

```
git add project-b/board/submission.json
git commit -m "A-project-b-AmbiguityCheck-1: document first ambiguous requirement"
```

3. Complete "**Ambiguity 2 - Explanation**". Save JSON again.
4. Commit with identifier **A-project-b-AmbiguityCheck-2**:

```
git commit -m "A-project-b-AmbiguityCheck-2: document second ambiguous requirement"
```

A.2 Conflict Resolution (1 required submission)

Identify any conflicting requests between stakeholders (e.g., User A wants X, but User B wants Y). Propose a solution.

1. Complete the **Conflict Resolution** field. Save JSON.
2. Commit with identifier **A-project-b-ConflictResolution**:

```
git commit -m "A-project-b-ConflictResolution: resolve stakeholder conflict with proposed solution"
```

A.3 Categorization: Top 3 Non-Functional Requirements

List the top 3 Non-Functional Requirements (NFRs) derived from the notes (e.g., security, performance, usability).

1. Complete NFR 1, 2, and 3. Save JSON.
2. After all NFR fields for **project-b** are complete:

```
git commit -m "A-project-b-Categorization: define 3 NFRs"
```

Part A for project-c

Complete the Analysis for **project-c** in the project folder `/project-c`.

Part B: Requirement Specification

Part B for project-a

A user story describes a system requirement from the perspective of a user.

Open `project-a/board/index.html` and switch to **Part B: Specification**. Create user stories in the **Standard Work** swimlane using:

As a **[Role]**, I want to **[Feature/Action]**, so that **[Benefit/Value]**.

- At least 5 distinct User Story tickets.
- At least 1 ticket with persona **Administrator/Staff**.
- At least 3 tickets with persona **Student**.
- **Acceptance criteria** in every story description (testable pass/fail checks).

Acceptance criteria are specific, testable conditions that define when a user story is considered complete. They should be written as clear pass/fail checks so that developers and testers can verify whether the feature works correctly.

Save JSON after a user story was added.

B.1 User Stories and Acceptance Criteria

Use a commit per story (US001 through US005) for project-a, for example:

```
git commit -m "B-project-a-US001: create user story 1"
git commit -m "B-project-a-US002: create user story 2"
```

Continue through **B-project-a-US005**. *Remember to save JSON after a user story was added.*

B.2 NFR Epics

A non-functional requirement describes how the system should behave rather than what function it performs. An epic is a large requirement or work item that groups several related tasks, stories, or improvements. In this assignment, NFR epics represent broad quality goals, such as improving system security, performance, or usability, that may later be broken down into smaller tickets.

1. In **project-a**, click **Add 3 NFRs as Epics**.
2. Save JSON and commit with identifier B-project-a-NFR-Epics:

```
git commit -m "B-project-a-NFR-Epics: add three NFR epic tickets"
```

Part B for project-b

Open `project-b/board/index.html` and switch to **Part B: Specification**. Create user stories in the **Standard Work** swimlane using:

As a **[Role]**, I want to **[Feature/Action]**, so that **[Benefit/Value]**.

- At least 5 distinct User Story tickets.
- At least 1 ticket with persona **Administrator/Staff**.
- At least 3 tickets with persona **Student**.
- **Acceptance criteria** in every story description (testable pass/fail checks).

Save JSON after a user story was added.

B.1 User Stories and Acceptance Criteria

Use a commit per story (US001 through US005) for project-b, for example:

```
git commit -m "B-project-b-US001: create user story 1"
git commit -m "B-project-b-US002: create user story 2"
```

Continue through **B-project-b-US005**. *Remember to save JSON after a user story was added.*

B.2 NFR Epics

1. In **project-a**, click **Add 3 NFRs as Epics**.
2. Save JSON and commit with identifier B-project-a-NFR-Epics:

```
git commit -m "B-project-a-NFR-Epics: add three NFR epic tickets"
```

Part B for project-c

Complete the Specification for **project-c** in the project folder /project-c.

Grading Rubric

Part	Criteria	Points
Part A: Analysis (30 pts) (10 pts per project)	Ambiguity Identification: Correctly identifies 2 vague requirements and explains why they are vague (e.g., missing metrics, unclear boundaries).	12
	Conflict Resolution: Identifies a logical conflict between stakeholders and proposes a realistic, actionable solution.	12
	NFR Identification: Correctly lists 3 Non-Functional Requirements (Security, Performance, etc.) derived from the context.	6
Subtotal		30
Part B: Specification (54 pts) (18 pts per project)	User Story Structure: Stories follow the strict "As a... I want to... So that..." format. (6 points per project)	18
	Persona Validity: Includes at least 1 Admin story and 3 Student stories as requested. (4 points per project)	12
	Acceptance Criteria: Stories include clear, testable checklists (Pass/Fail conditions). (3 points per project)	15
	3 NFRs as Epics: Requirements are added and easy to read/understand. (3 points per NFR)	9
Subtotal		54
Professionalism (16 pts)	Traceability (commit identifiers + JSON): All required project-scoped tokens and snapshots are present.	8
	Professionalism and repository quality: Clear commits, organized structure.	8
Subtotal		16
TOTAL		100

Good luck!

Project Case Studies

Project A: "Pathfinder" (Intelligent Academic Advising)

Context: Students are struggling to graduate on time because they take the wrong prerequisites or can't find open slots for mandatory courses. The university wants an automated roadmap builder.

Raw Elicitation Notes:

- **Interview with Head of Advising:** "I spend 6 hours a day answering emails asking 'Can I take Course B if I haven't done Course A?' The system needs to stop them from even asking. Also, I want to be able to override the system manually if a student has special permission."
- **Focus Group (Freshmen):** "The course catalog is ugly. I just want to drag and drop classes into a calendar to see if they fit. I want the app to warn me if I'm choosing a class that starts at 8:00 AM because I will definitely oversleep."
- **IT Security Officer:** "Student data is sensitive. Under no circumstances should a student be able to see another student's schedule. Also, the system must log every time an advisor manually overrides a prerequisite for audit purposes."
- **Feedback from Registrar:** "The system needs to link to the live database. If a class is full, the student shouldn't be able to add it to their roadmap. But the Advising Head said they want students to be able to plan 4 years ahead, even if we don't know if the class is full yet. That sounds contradictory."

Project B: "CampusLoop" (Circular Economy Marketplace)

Context: At the end of every semester, dumpsters are full of good textbooks, mini-fridges, and furniture. Meanwhile, incoming students buy everything new. The Student Union wants a peer-to-peer marketplace app.

Raw Elicitation Notes:

- **Student Union President:** "This needs to be exclusive. Only people with a valid '.edu' email address can create an account. We don't want random people from the city coming on campus to buy cheap couches."
- **Interview with Campus Police:** "We are worried about safety. We want the app to suggest 'Safe Exchange Zones' (like the library lobby) for all physical meetups. The app should force users to select a safe zone before they can chat."
- **Survey Response (Senior Student):** "I want to sell my textbooks, but I hate typing in the ISBN and Title. Can I just scan the barcode with my phone camera? Also, I want to be able to auction items, not just set a fixed price."
- **University Legal Team:** "The university cannot handle any money. We are not a bank. The app cannot process payments. Students have to pay each other via Venmo or cash. But the Student Union President said they want to take a 2% transaction fee to fund club activities. We need to figure that out."

Project C: "UniDine" (Mobile Cafeteria Ordering)

Context: The lunch rush at the main cafeteria is chaotic. Lines are 20 minutes long, and students with allergies often hold up the line asking about ingredients.

Raw Elicitation Notes:

- **Cafeteria Manager:** "We need to reduce the queue. I want students to order food on their phone while they are walking from class. The kitchen needs a screen that shows orders as they come in."
- **Interview with Chef:** "I need to be able to mark an item as 'Sold Out' instantly. Yesterday, people kept ordering the vegan wrap for 10 minutes after we ran out. It was a nightmare."
- **Student with Nut Allergy:** "I need to filter the entire menu. If I check 'No Peanuts', I don't even want to see the peanut butter cookie on the screen. It gives me anxiety."
- **IT Infrastructure Team:** "The cafeteria basement has terrible Wi-Fi. The kitchen display system needs to be able to work offline for at least 10 minutes without crashing if the internet cuts out. It should sync up once the connection is back."
- **Marketing Dept:** "We want to send push notifications to students at 11:30 AM offering them a discount on the daily special."

Commit Identifier Reference

Every commit message must include the exact token for the project and task. Replace `<project>` with **project-a**, **project-b**, or **project-c**. Example: A-project-a-AmbiguityCheck-1: clarified prerequisite override rule.

Identifier Token Pattern	When to Commit
A- <code><project></code> -AmbiguityCheck-1	After Ambiguity 1 explanation is saved.
A- <code><project></code> -AmbiguityCheck-2	After Ambiguity 2 explanation is saved.
A- <code><project></code> -ConflictResolution	After conflict resolution solution is saved.
A- <code><project></code> -Categorization	After 3 NFRs are entered.
B- <code><project></code> -US001	After creating user story 1 (increment story number).
B- <code><project></code> -NFR-Epics	After generating three NFR epic tickets.

Required `<project>` values: **project-a**, **project-b**, **project-c**.